

THE INSTITUTE OF FINANCE MANAGEMENT



SOFTWARE REQUIREMENT SPECIFICATION (SRS) FOR:

RENT PAY: DIGITAL RENTAL PROPERTY PAYMENT AND
MANAGEMENT PLATFORM.

PROJET GROUP NO. 4.

S/N	STUDENT NAME	REGISTRATION NUMBER
01.	ASIA OMARY UWESU	IMC/BIT/2414552
02.	DALIUS NYEMENOHI JOVENARY	IMC/BIT/2426689
03.	RAMADHANI SELEMANI SAIDI	IMC/BIT/2425288
04.	DANIEL JAPHET MAFURU	IMC/BIT/2423039
05.	MOHAMED ALLY MKELANGELA	IMC/BIT/2421970

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Purpose

The main purpose of this Software Requirements Specification (SRS) document is to provide a clear and complete description of the Rent Pay System (RentPay). This document lists all the functional and non-functional requirements of the system. It serves as a strict guide for the development team, project managers, and school examiners to understand what the system does, how it works, and how it handles rental transactions securely.

1.2 Scope

The Rent Pay System (RentPay) is a web-based electronic commerce platform designed to modernize the rental property market in Tanzania by replacing old, paper-based tracking methods with a centralized digital system. The platform bridges the gap between property owners and renters by bringing room hunting, rent collection tracking, and basic tenant-landlord communication into a single online environment.

1.2.1 In Scope

The system will fully support and provide the following core capabilities:

- **Online Property Search:** Prospective tenants can easily browse and search through public rental listings based on location and price points without needing to hire physical brokers.
- **Digital Payment Automation:** The software connects directly to the Vodacom M-Pesa Daraja API to trigger automated payment screens on user phones, allowing tenants to pay rent digitally.
- **Financial and Expense Tracking:** Landlords can easily track their rental income, monitor outstanding tenant bills, and log property expenses to see their net profits clearly.

- **Maintenance Communication:** Tenants can submit formal text reports about structural or utility problems in their rooms, and landlords can update them on the status of the repairs.
- **System Administration:** Administrative staff have access to a central console to oversee user safety, verify uploaded business files, and monitor transaction balances.

1.2.2 Out Scope

To ensure the application can be successfully completed within its academic timeline, the following features are strictly excluded from this project:

- **Real Currency Handling:** The system functions entirely within a sandbox test environment; no real currency or actual legal money will be processed.
- **Standalone Mobile Applications:** The application is built strictly for web browsers and will not include separate downloadable smartphone apps for Android or iOS.
- **Legal Arbitration and Disputes:** The system operates as a tracking utility and will not provide formal legal services, contract signatures, or court-level dispute resolution for user conflicts.
- **Artificial Intelligence Tools:** The software will not feature complex AI systems for predicting house prices or running automated background checks on tenants.
- **Live Multimedia Communication:** Features such as real-time instant messaging, video calling, and live virtual house tours are completely excluded.
- **Government Database Syncing:** The software acts independently and will not connect to live national property or identity registries.

1.3 Product perspective

The Rent Pay System is a standalone software application created to bridge the gap between landlords and tenants. It replaces manual, paper-based cash collections with digital tracking. The application relies heavily on third-party integration, specifically connecting directly with the Vodacom M-Pesa Daraja Sandbox API to process mobile financial transactions in real-time.

1.4 Product functions

The core functions of the Rent Pay System include:

- **User Authentication and Profiling:** Registering and logging in Tenants, Landlords, and Admins safely.
- **Property Management:** Allowing landlords to add, modify, or remove available rooms and property descriptions.
- **Invoice Generation:** Automatically creating billing payloads based on property prices and lease agreements.
- **Payment Escrow Processing:** Triggering the M-Pesa STK push and holding paid funds securely in an escrow vault until the user moves in.
- **Dispute and Refund Management:** Allowing tenants to report issues and giving admins the power to refund money or release funds to the landlord.

1.5 User characteristics

The Rent Pay System interacts with four specific types of human users, each having different technical backgrounds, roles, and access rights. Below is a detailed description of their characteristics based directly on the system's use cases:

1.5.1 Visitor

- **Description:** An unregistered or anonymous guest browsing the public website.

Anyone with basic phone or computer skills who knows how to open a web link.

- **Role in the System:** They have the lowest level of access. According to the use case diagram, their actions are limited strictly to public informational pages. They use the system to view the home overview, browse property listings, check contact details, and create a formal account through the registration portal.

1.5.2 Tenant

- **Description:** A registered user looking for a house or occupying a rental unit.

Simple smartphone or web browser experience. They are familiar with using standard consumer apps and mobile money services like M-Pesa.

- **Role in the System:** Once logged in, their active characteristics center around finding and maintaining a tenancy. They use the platform to search and filter through property categories, submit formal housing applications, view digital invoices, trigger mobile wallet rent payments, and open maintenance tickets to report damaged room utilities.

1.5.3 Landlord

- **Description:** A property owner or business manager running a rental housing business.

Low to medium computer skills. They might not be advanced IT professionals, so they require clean forms and straightforward text menus.

- **Role in the System:** They manage the business supply side. Their system tasks include setting up a verified seller profile, creating and updating rental house items, reviewing tenant applications to accept or reject them,

managing structural repair statuses, and analyzing automated net income spreadsheets.

1.5.4 Admin (System Administrator)

- **Description:** The internal technical operator responsible for keeping the platform safe, secure, and running smoothly.
Highly trained IT professional with strong knowledge of system configuration, security rules, and digital accounting.
- **Role in the System:** They have full backend authority. Their operational characteristics involve checking landlord verification documents (like BRELA or PDPC uploads), monitoring the security of the escrow payment balances, executing financial refunds or payouts during user disputes, banning bad profiles, and extracting system-wide operation reports for audits.

1.6 Limitations

- **Network Dependency:** Users must have a stable internet connection and active mobile network service to receive payment prompts.
- **Payment Boundaries:** The system is limited strictly to mobile money integrations (like the M-Pesa Sandbox) and does not support cash or physical credit cards.
- **Verification Delays:** Landlord approval is not instant; it depends on manual verification by a human admin.

1.7 Assumptions and dependencies

- **Assumptions:**
 - It is assumed that all landlords will provide genuine registration and property ownership documents.

- It is assumed that tenants have a mobile phone capable of receiving SIM Toolkit (STK) notifications.
- **Dependencies:**
 - The system heavily depends on the continuous availability of the Vodacom M-Pesa Daraja API gateway. If the mobile money API goes offline, the payment portal will not function.
 - The app also depends on secure hosting servers and database availability to prevent data loss.

1.8 Definitions

- **RentPay Platform:** The core web system handling the application logic and connecting tenants, landlords, and administrators.
- **Escrow:** A secure financial arrangement where a third party (our system) holds money safely until a specific condition is met (such as the tenant confirming they have successfully moved into the room).
- **Database:** A digital storage space where all system information, user records, and transaction logs are safely kept.

1.9 Acronyms and abbreviations

- **SRS:** Software Requirements Specification
- **STK Push:** Sim Toolkit Push (An automatic M-Pesa PIN pop-up on a user's phone to process a payment)
- **API:** Application Programming Interface
- **DB:** Database
- **BRELA:** Business Registration and Licensing Agency
- **PDPC:** Personal Data Protection Commission

3. FUNCTIONAL REQUIREMENTS

3.1 User Account & Access Control

FR-1: User Registration and Authentication

- **Description:** The system shall let new users create an account and let existing users log in securely.

Think of this as the front door. The system needs to know who is a Tenant, who is a Landlord, and who is an Admin because they all see different dashboards. It must validate emails/phone numbers and have a "Forgot Password" option.

FR-20: Administrative Management

- **Description:** The system shall provide a backend dashboard for system administrators to oversee the platform.

The Admin is the supervisor. If a landlord posts something illegal, or if there is a payment dispute, the Admin must have the power to view user profiles, block accounts, monitor escrow status, and check overall platform activities.

3.2 Property & Room Inventory (The Product Catalog)

FR-2: Property Listing Management

- **Description:** The system shall allow landlords to add, edit, or delete their property details.

In e-commerce, a seller uploads products. Here, the landlord uploads their house/apartment. The system must let them upload pictures, write a description, set the location, and switch the status between "Available" or "Fully Booked."

FR-3: Room Management

- **Description:** The system shall allow landlords to split a single property into individual rental units or rooms.

An apartment building has different rooms. This feature lets the landlord specify details for each room—like Room 101 is a single bedroom for \$200, while Room 102 is a double bedroom for \$350.

3.3 Discovery AND Exploration (The Shopping Stage)

FR-4: Property Search and Filtering

- **Description:** The system shall allow browsers/customers to search for properties based on specific criteria.

Users shouldn't have to scroll blindly. The system must have a search bar and filters so a student can type "Dar es Salaam," filter by "Price: Under \$150," and instantly see matching rooms.

FR-5: Property Comparison

- **Description:** The system shall allow users to select multiple properties and view them side-by-side.

Just like comparing two phones on Amazon, a tenant should be able to click "Compare" on three different rooms to see their prices, locations, and amenities (like Wi-Fi or AC) right next to each other on one screen.

FR-6: Property Viewing

- **Description:** The system shall display a comprehensive detail page for any selected property or room.

When a user clicks on a search result, the system must open a dedicated page showing high-resolution images, video tours, full lists of rules, and current availability status.

3.4 Booking and Order Processing

FR-7: Booking Request Management

- **Description:** The system shall allow customers to submit an official request to reserve a room.

This is the "Add to Cart / Proceed to Checkout" step. The tenant clicks "Book Now," selects their intended move-in date, and submits the request to the system.

FR-8: Booking Approval Management

- **Description:** The system shall notify landlords of new booking requests and allow them to accept or reject them.

Landlords get a notification saying "Asia wants to book Room 10A." The system must give the landlord two clear buttons: **Approve** or **Reject**.

3.5 Digital Legalities (The Innovation Core)

FR-9: Digital Rental Agreement Generation

- **Description:** The system shall automatically create a legal rental contract once a booking is approved.

Instead of making people meet in person to write a contract, the system pulls data from the database (Tenant Name, Landlord Name, Room Price, Dates) and automatically creates a digital contract document.

FR-10: Agreement Acceptance and Digital Signing

- **Description:** The system shall provide a way for both parties to review and legally sign the agreement online.

The system displays the generated contract to both the landlord and tenant. They must be able to click a check-box or type their name to "Digitally Sign" it. The system then locks the document so it cannot be altered.

3.6 Secure Financial Transactions (E-Commerce Trust)

FR-11: Escrow Payment Service

- **Description:** The system shall hold the tenant's money in a temporary, secure account before giving it to the landlord.

This prevents fraud. When the tenant pays, the money does not go straight to the landlord's pocket. The system holds it safely in the middle (Escrow) so the landlord knows the tenant has paid, and the tenant knows the landlord can't run away with the money before they move in.

FR-12: Online Rent Payment

- **Description:** The system shall process electronic payments through integrated payment

gateways. The system must show a payment checkout page where tenants can pay their rent using mobile money (like M-Pesa, Airtel Money) or bank cards.

FR-13: Escrow Release Management

- **Description:** The system shall transfer funds from the escrow account to the landlord's account once move-in conditions are met.

Once the agreement is signed and the tenant confirms they have successfully moved into the room, the system changes the transaction status and automatically sends the held money to the landlord.

FR-14: Payment Status Tracking

- **Description:** The system shall display the live status of any payment transaction to both users.

Both users should see a progress bar or label showing exactly where the money is: Pending, In Escrow, Released, or Failed.

FR-15: Receipt Generation

- **Description:** The system shall generate a downloadable, uneditable receipt after a successful transaction.

As soon as the payment clears, the system automatically builds a PDF receipt with a unique transaction ID, date, and amount, and saves it in both the tenant's and landlord's payment history.

3.7 Communication and Post-Occupancy Management

FR-16: Notification Management

- **Description:** The system shall send automated alerts to users when important actions occur.

The system acts like an assistant. It sends internal alerts, SMS, or emails whenever a booking is requested, an agreement is signed, rent is due, or money is released.

FR-17: Maintenance Request Management

- **Description:** The system shall allow tenants to report issues and let landlords update the repair status.

If a pipe bursts, the tenant goes to their dashboard, types "Bathroom leaking," and hits submit. The landlord sees this request and can update it from "Pending" to "In Progress" to "Fixed."

FR-18: Tenant and Landlord Communication

- **Description:** The system shall host an internal chat feature for direct text messaging.

To avoid sharing personal phone numbers too early, the system provides a simple, built-in chat box where landlords and tenants can text each other about property details or move-in times.

3.8 Business Intelligence

FR-19: Reporting and Analytics

- **Description:** The system shall compile transaction and usage data into visual reports.

Landlords need to see how much money they made this month. The system calculates their totals and displays simple charts showing rental income, active bookings, and occupied vs. vacant rooms.

4. NON – FUNCTIONAL REQUIREMENTS

4.1 Speed Operations

NFR - 01 Performance Requirements (Speed)

This is all about **speed**. Users hate slow websites. Your requirements state that when a tenant clicks "Search," the results must appear within 3 seconds. When they log in, it shouldn't take more than 5 seconds. If 100 students in your hostel use Rent Pay at the exact same time, the server shouldn't crash or slow down.

NFR – 04 Availability Requirements (Uptime)

This means the system must be **alive and working** 24/7. Rent doesn't wait, and a tenant might want to pay at 2:00 AM. Your target is **99% availability**, meaning the site should almost never go offline, except for brief, announced late-night maintenance.

4.2 Trust, Safety & Records

NFR – 02. Security Requirements (Protection)

Since Rent Pay handles real money, it is an attractive target for hackers. This requirement ensures that passwords are never saved as plain text (they must be scrambled/hashed in the database). It also means a tenant cannot hack into the admin panel or alter a digital lease agreement after it is signed.

4.3 Data Integrity Requirements (Accuracy)

Data integrity means **no mistakes in the database**. For example, if a tenant pays rent once, the system must never accidentally create a duplicate record showing they paid twice. Every contract and payment must have a unique ID so nothing gets mixed up or corrupted.

NFR – 03. Auditability Requirements (Tracking)

The system must keep a hidden **activity log book**. If a landlord claims, *"I never received the money,"* or a tenant claims, *"I moved in on Friday,"* the Admin must be able to look at the history logs to see the exact second the button was pressed. This is your proof for dispute resolution.

NFR – 05. Legal and Compliance Requirements (Rules)

Your system must follow the law. Before anyone registers, they must check a box agreeing to your "Terms and Conditions." Furthermore, financial records must be stored safely for years because tax authorities or courts require clear records for housing disputes.

4.4 User Experience & Reliability

NFR – 05. Usability Requirements (User-Friendliness)

The system must be **easy to use without a manual**. If a landlord is older and not tech-savvy, the buttons should be big, clear, and labeled "Add Room" instead of using complex technical jargon. Error messages should say *"Please enter a valid phone number"* instead of displaying an ugly database error code.

NFR – 06. Reliability Requirements (Consistency)

Reliability means the system is **dependable**. If the internet cuts out exactly while a tenant is transferring escrow funds, the system must handle the failure gracefully. It should either complete the transaction safely or cancel it entirely—it must never leave the money stuck in limbo or corrupt the database.

4.5 Growth and Technology Environment

NFR – 07. Scalability Requirements (Growth)

Your system shouldn't break when it gets popular. Today it might handle 50 rooms in one hostel, but next year it should be able to handle 50,000 rooms across the whole city without needing a complete system rewrite. It also means the architecture is designed to support a mobile app later.

NFR – 08. Maintainability Requirements (Easy to Fix)

The code must be clean and organized. If you graduate and hand this project over to another IT student, they should be able to look at your code, understand it quickly, and add a new feature (like an automated SMS alert) without accidentally breaking the payment system.

NFR – 09. Portability Requirements (Devices)

The application shouldn't care what device the user has. It must load correctly and look good whether a user opens it on Google Chrome on a laptop, Safari on an iPhone, or Mozilla Firefox on a desktop.

NFR – 10. Compatibility Requirements (Integrations)

Rent Pay does not work completely alone. It needs to play nice with other software. It must smoothly connect to a MySQL database, an email server

to send alerts, and payment APIs (like mobile money systems) without conflicting configurations.

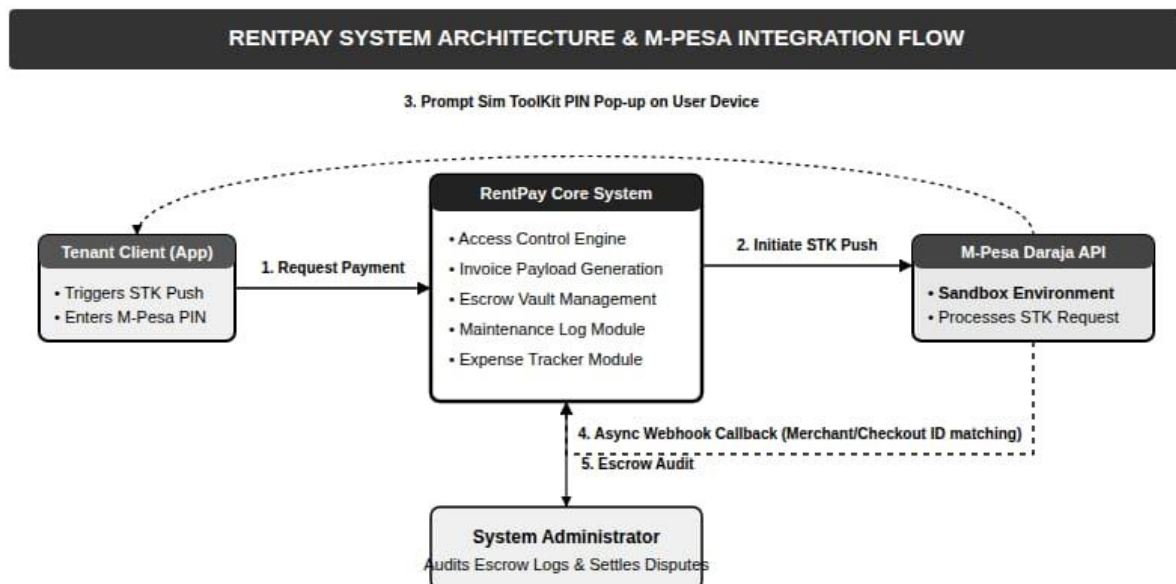
5. OTHER REQUIREMENTS

5.1 APPENDIX A: ANALYSIS MODELS.

5.1.1 High system architecture and context

This section will include the following analysis models:

5.1.1.1 SYSTEM ARCHITECTURE DIAGRAM



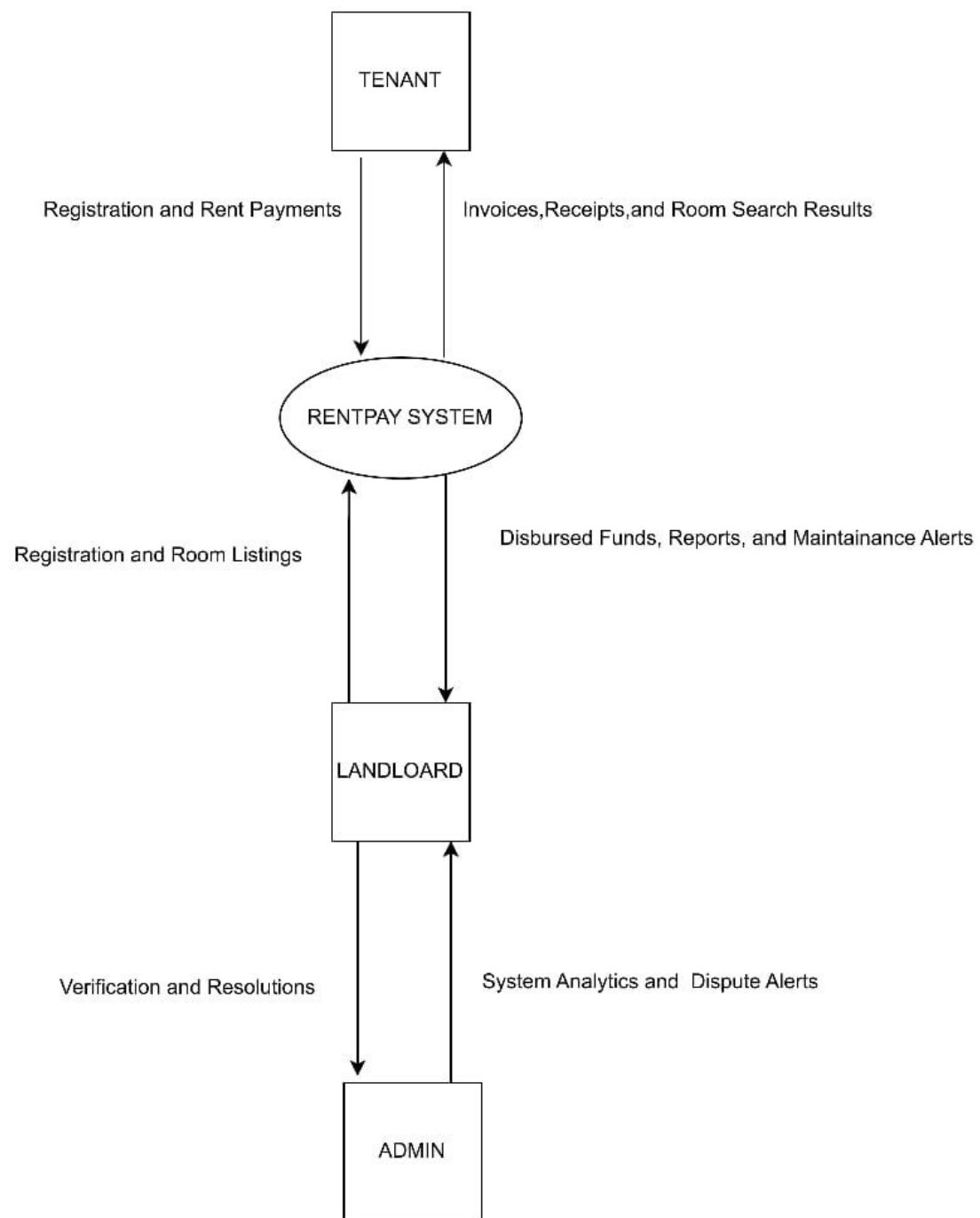
Objective: Shows the overall high-level technical structure of the application and how components talk to each other.

Explanation:

- The **Tenant Client App** is the interface where users initiate payments and enter their M-Pesa PINs.

- The **RentPay Core System** acts as the main backend server, handling access control, invoice data, the escrow vault, and maintenance logs.
- The **M-Pesa Daraja API (Sandbox)** processes the SIM toolkit (STK) push notifications and returns transaction status confirmations using asynchronous webhooks.

5.1.1.2 CONTEXT DATAFLOW DIAGRAM(Level 0 DFD)

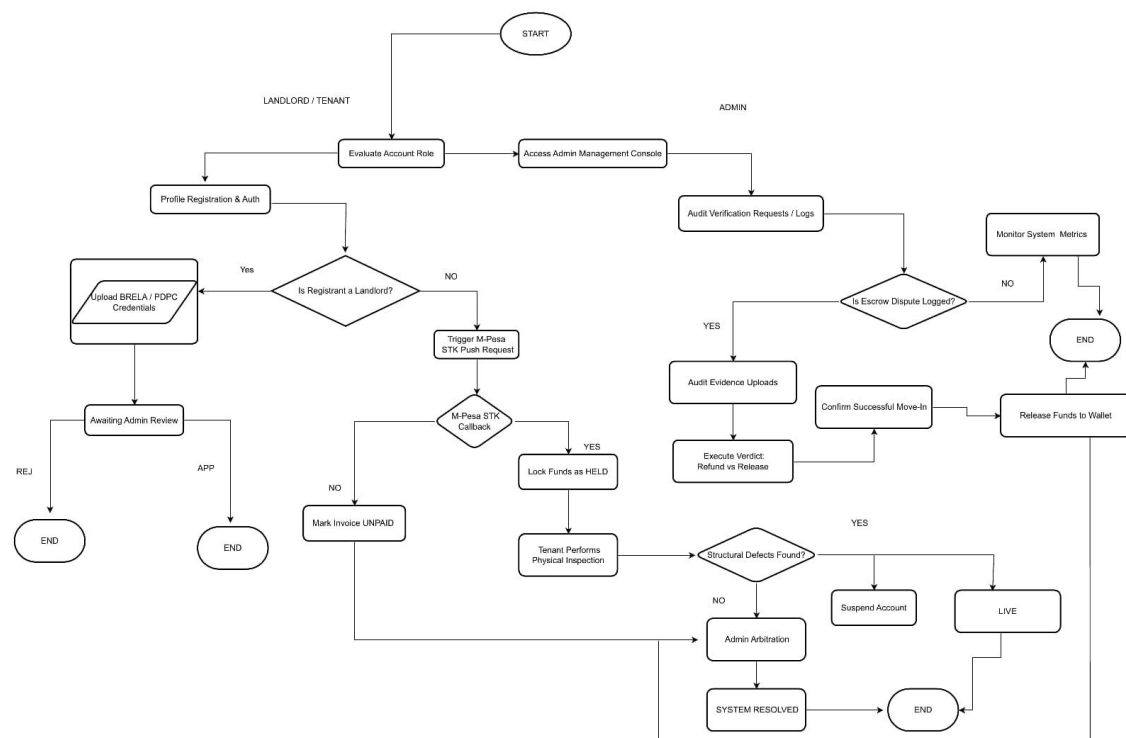


Objective: Defines the system boundaries, showing what flows into and out of the RentPay platform as a single whole process.

Explanation:

- The **Tenant** sends registration details and rent payments into the system, and receives invoices, receipts, and property search results.
- The **Landlord** inputs property listings and receives disbursed funds, reports, and maintenance updates.
- The **Admin** sends verifications and dispute resolutions to the platform, while receiving system analytics data.

5.1.1.3 SYSTEM FLOWCHART



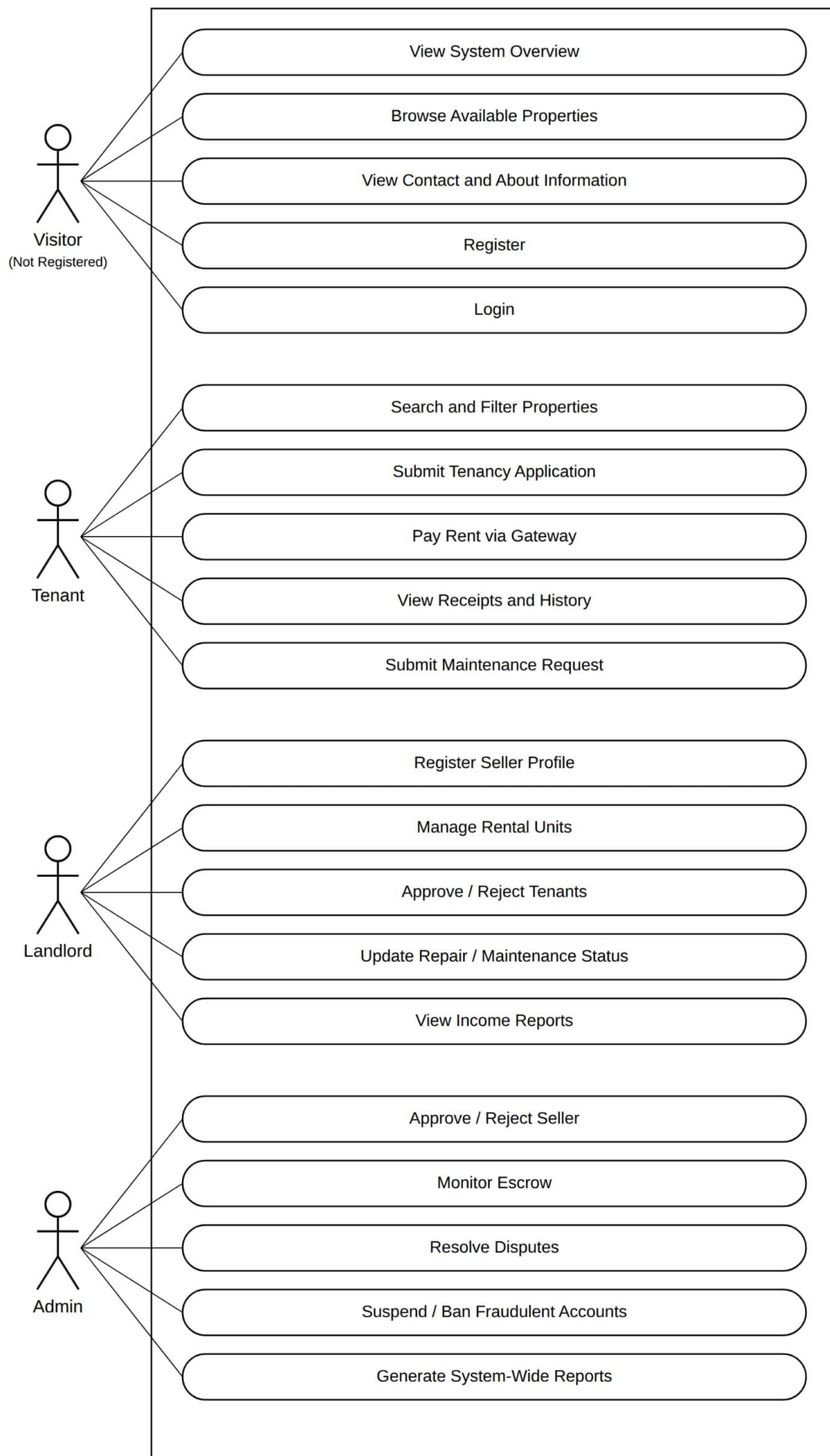
Objective: Displays the chronological, step-by-step business logic from the start of an operation to its final resolution.

Explanation:

- The process splits based on the user's role. Tenants and Landlords complete registration and profiling.
- If the user is a Landlord, they upload BRELA or PDPC credentials for Admin review, which leads to approval or rejection.
- If the user is a Tenant making a payment, an M-Pesa STK Push is triggered. If successful, funds are held securely in an escrow state while the tenant inspects the room.
- Admins use an administrative console to handle disputes, examine evidence, and execute verdicts to either refund the money or release it to the landlord's wallet.

5.1.2 PROCESS AND BEHAVIORAL MODELING

5.1.2.1 USECASE DIAGRAM



Objective of usecase diagram

This diagram shows a simple map of all the different users (Actors) in the RentPay System and exactly what tasks (Use Cases) they are allowed to perform on the platform.

Explanation of Actors and Core Tasks:

1. Visitor (Not Registered User)

This is a guest user who has not created an account yet so he /she is not a tenant either. They can perform basic public activities:

- **View System Overview:** Look at the main homepage to see how RentPay works.
- **Browse Available Properties:** Look around at different rooms and rental houses posted on the website.
- **View Contact and About Information:** Read details about the platform developers or contact customer care.
- **Register or Login:** Create a fresh account or sign in to open more system features.

2. Tenant An authenticated renter who is looking for or occupying a room. Their core system use cases are:

- **Search and Filter Properties:** Find rooms easily by filtering by location, price range, or house type.
- **Submit Tenancy Application:** Formally apply to move into a selected house or room listing.
- **Pay Rent via Gateway:** Trigger the integrated mobile payment (M-Pesa API) to pay rent securely into the escrow vault.

- **View Receipts and History:** Look at their past financial ledger statements and download digital payment receipts.
- **Submit Maintenance Request:** Open a structural repair ticket if there is a broken pipe, light, or wall defect in their room.

3. Landlord

A property owner running a rental business through the platform. Their active use cases are:

- **Register Seller Profile:** Upload legal verification forms to create a licensed landlord profile.
- **Manage Rental Units:** Post new rooms, update monthly pricing tiers, or delete houses that are fully occupied.
- **Approve or Reject Tenants:** Review applications sent by tenants and decide who gets the room.
- **Update Repair or Maintenance Status:** Change tracking details on active repair logs to let tenants know fixes are on the way.
- **View Income Reports:** Look at clean financial sheets detailing their net monthly payouts.

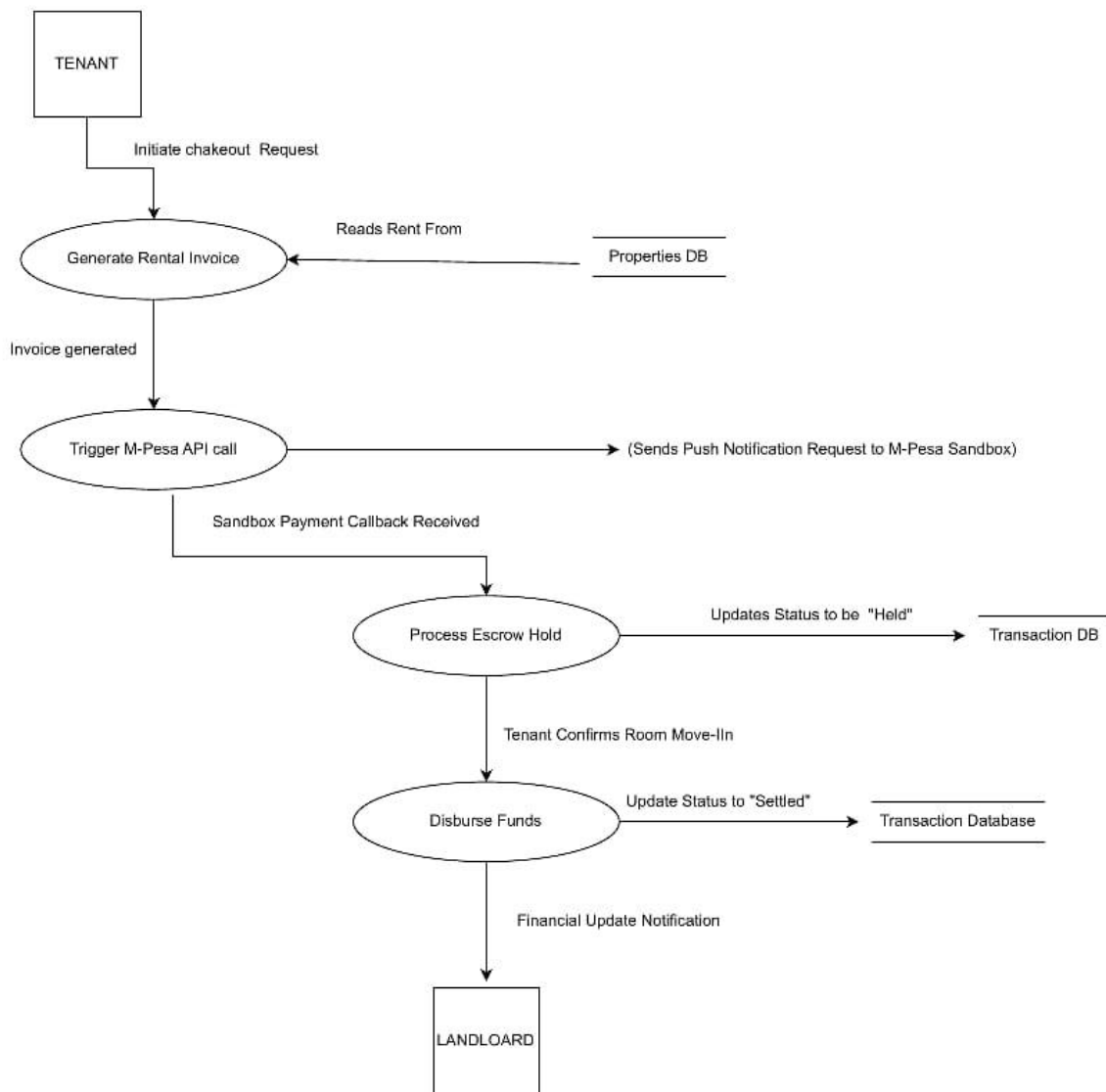
1. Admin (System Administrator)

The backend operator managing platform safety and balances. Their essential use cases include:

- **Approve or Reject Seller:** Manually check landlord submissions (BRELA or PDPC files) to confirm if their company is legal.

- **Monitor Escrow:** Keep track of funds locked in the digital vault to prevent early releases or missing cash.
- **Resolve Disputes:** Look over case evidence when a tenant submits a complaint, deciding if they should process a refund or send the money to the landlord.
- **Suspend or Ban Fraudulent Accounts:** Block bad landlords or fake profiles immediately to protect other users.
- **Generate System-Wide Reports:** Collect system metrics, traffic data, and total platform transactions for business reviews.

5.1.2.1 LEVEL 1 DATA FLOW DIGRAM(DFD)

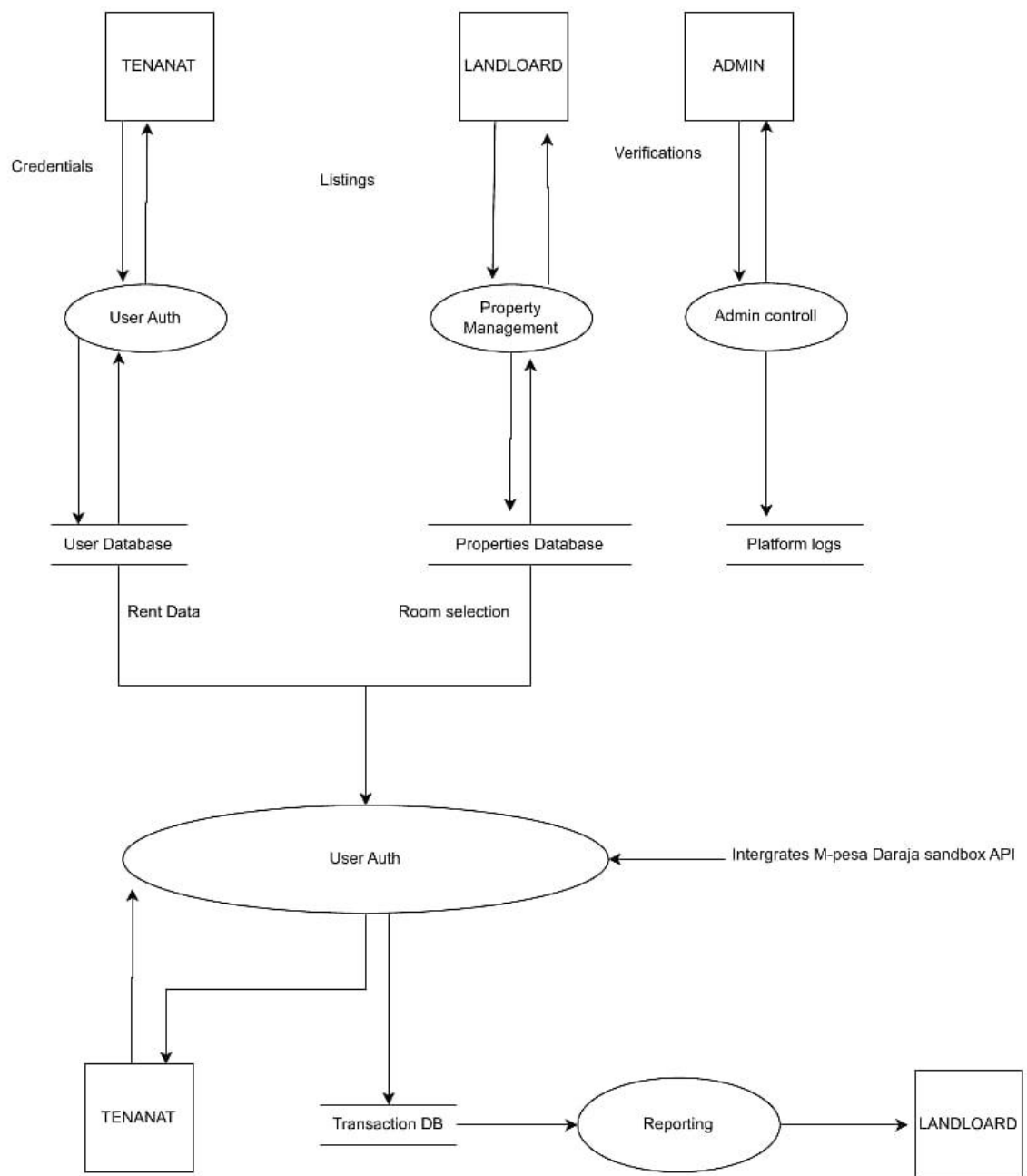


Objective: This diagram breaks the main system down into its core functional sub-processes and shows how data is stored.

Explanation:

- Users interact directly with sub-processes like **User Auth**, **Property Management**, and **Admin Control**.
- These processes interact with individual storage areas: the **User Database**, **Properties Database**, and **Platform Logs**.
- The primary interaction links User Auth and Property Management down into a central process integrated with the M-Pesa Daraja Sandbox API, which populates the **Transaction DB** for **Reporting**.

5.1.2.2 LEVEL 2 DATAFLOW DIAGRAM (DFD)

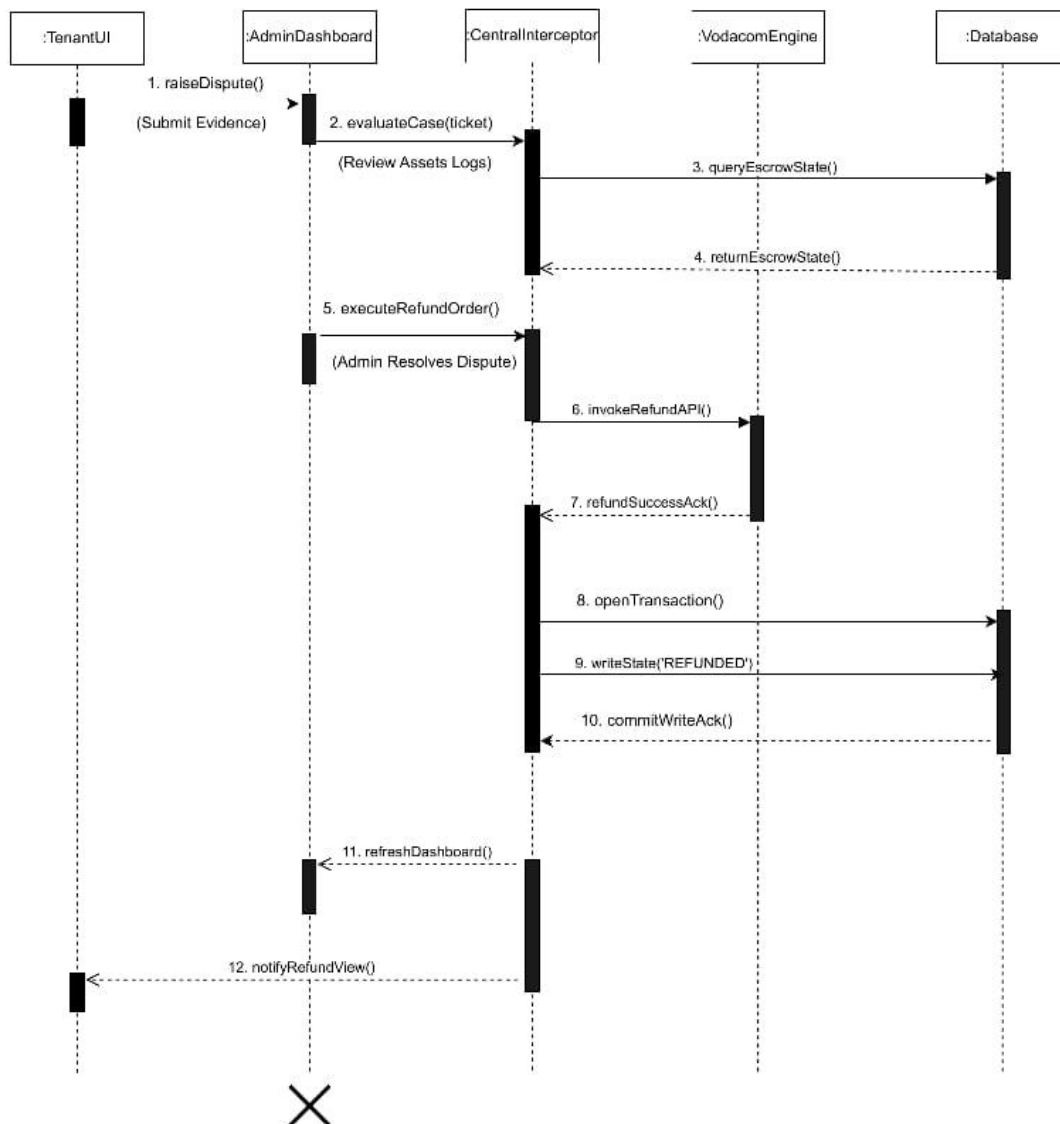


Objective: Takes the core payment sub-process from Level 1 and magnifies it to show exact internal data paths.

Explanation:

- A Tenant triggers an initial checkout request which calls the **Generate Rental Invoice** process, reading data straight from the Properties DB.
- Once the invoice payload is generated, it triggers the **M-Pesa API Call** to send a push notification request.
- When the payment callback is received, the system passes the data to **Process Escrow Hold**, changing the transaction status to "Held".
- After the room move-in is confirmed, the **Disburse Funds** process updates the Transaction DB status to "Settled" and fires a financial update notification to the Landlord.

5.1.2.3 SEQUENCE DIAGRAM

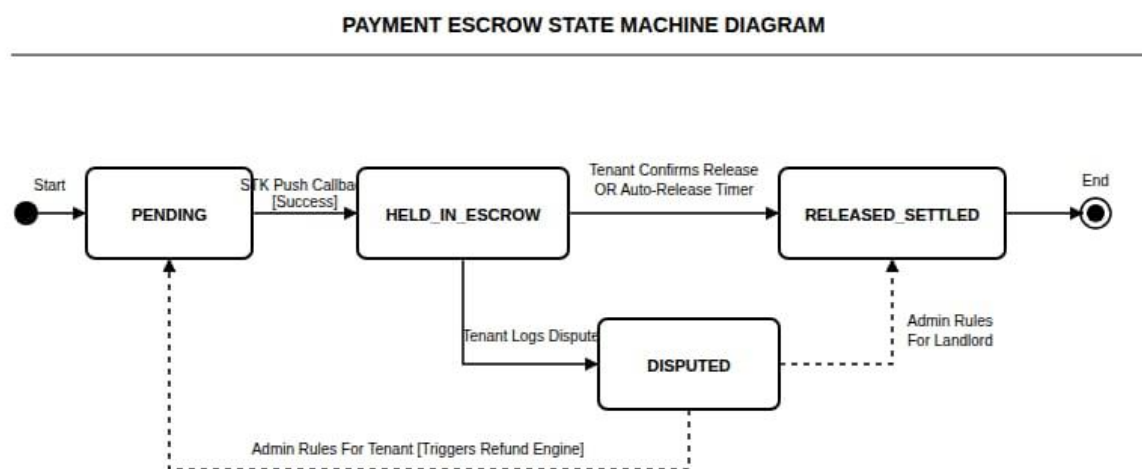


Objective: Illustrates time-ordered step-by-step interactions between different system components during a dispute refund.

Explanation:

- The TenantUI raises a dispute and submits evidence to the AdminDashboard.
- The admin dashboard routes the case to a CentralInterceptor which queries the Database to check the current escrow state.
- Once verified, the admin dashboard executes a refund order. The CentralInterceptor invokes the refund API on the VodacomEngine.
- Upon receiving a success acknowledgement, a database transaction opens to write the status as 'REFUNDED'. The change is committed, refreshing the admin dashboard and alerting the tenant view.

5.1.2.4 STATE MACHINE DIAGRAM



Objective: Tracks the lifecycle phases and behavioral changes of a transaction inside the payment escrow vault.

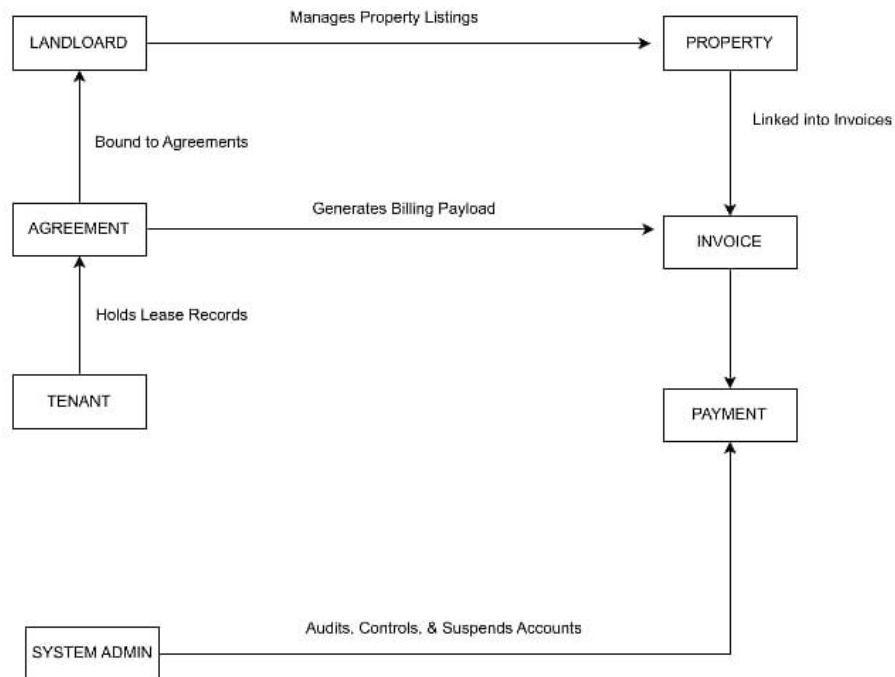
Explanation:

- The transaction starts in a **PENDING** state.
- Receiving a successful M-Pesa STK push callback transitions the transaction to **HELD_IN_ESCROW**.

- Under normal circumstances, a tenant confirmation or an expiration timer moves the money to **RELEASED_SETTLED**, ending the lifecycle.
- If a tenant files a complaint, the state transitions to **DISPUTED**. From there, admin rules determine whether it triggers the refund engine to go back to **PENDING** or settles out to the landlord under **RELEASED_SETTLED**.

5.1.3 STRUCTURAL AND DATABASE DESIGN

5.1.3.1 CONCEPTUAL ENTITY RELATIONSHIP DIAGRAM(ERD)

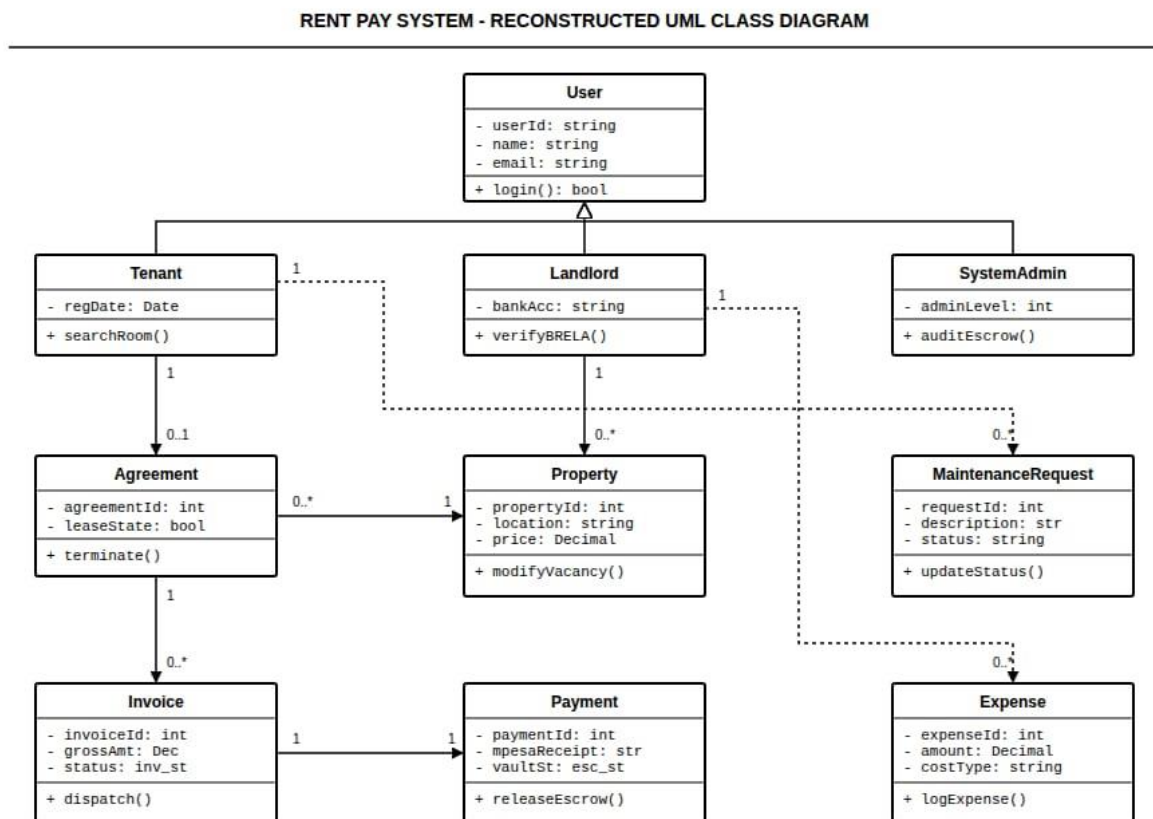


Objective: Presents a simple, business-friendly map of high-level concepts and how they relate without technical specifics.

Explanation:

- A **Tenant** holds lease records which are bound to an **Agreement**.
- A **Landlord** signs the **Agreement** and manages **Property** listings.
- The **Agreement** and **Property** concepts combine to generate billing data for an **Invoice**, which leads directly to a **Payment**.
- A **System Admin** is an external entity that audits, controls, and suspends the accounts tied to these components.

5.1.3.2 UML CLASS DIAGRAM

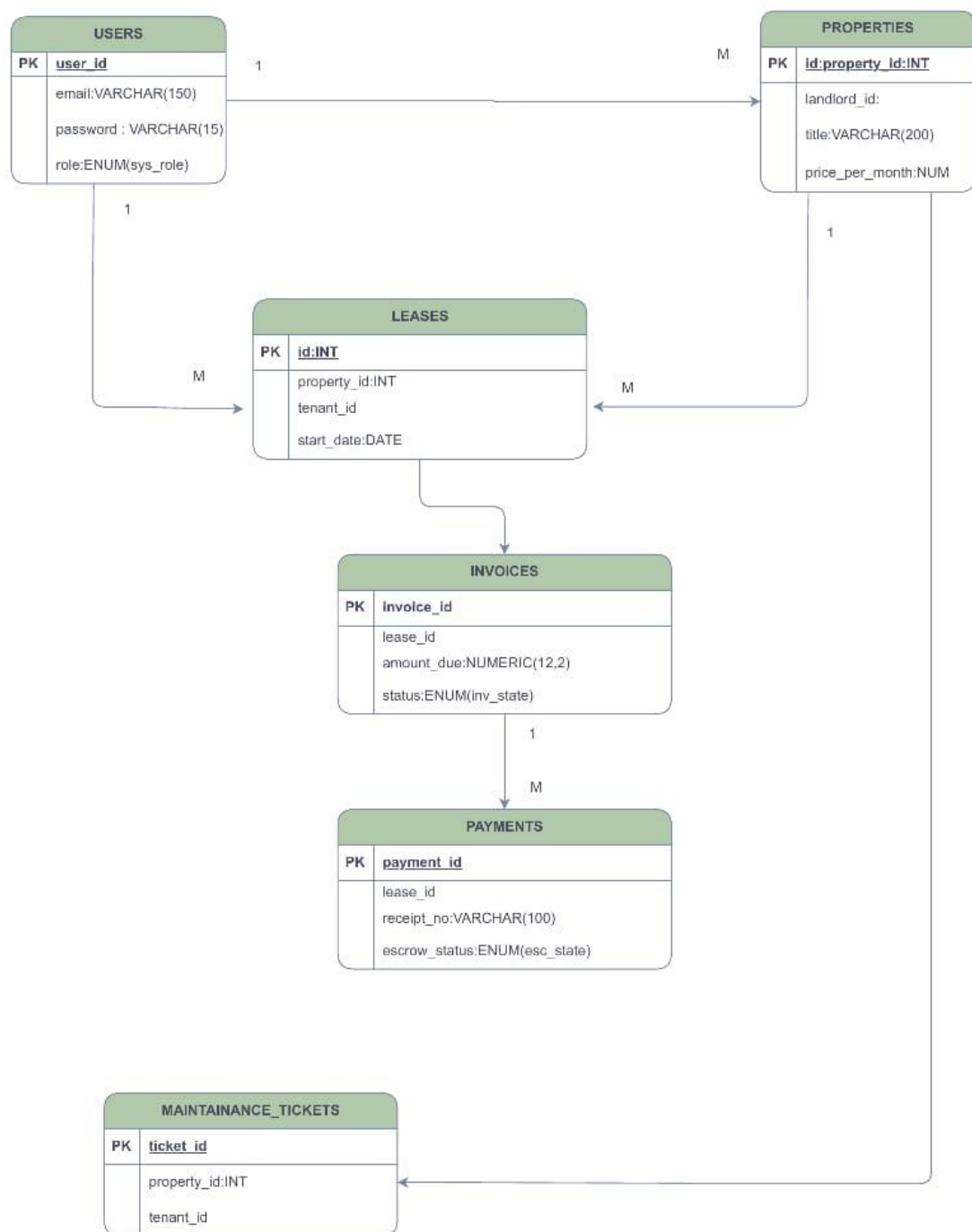


Objective: Displays the structural design of the backend code, including classes, attributes, methods, and inheritance.

Explanation:

- There is a primary parent class called User that contains shared traits like ID, name, email, and the login() action.
- Tenant, Landlord, and SystemAdmin all inherit directly from the User class.
- The structural map shows clear associations: a Tenant signs an Agreement, an Agreement produces an Invoice, and an Invoice triggers a Payment.
- Landlord connects to Property and Expense classes, while SystemAdmin links directly to handling MaintenanceRequest entities.

5.1.3.3 LOGICAL ENTITY RELATIONSHIP DIAGRAM(ERD)

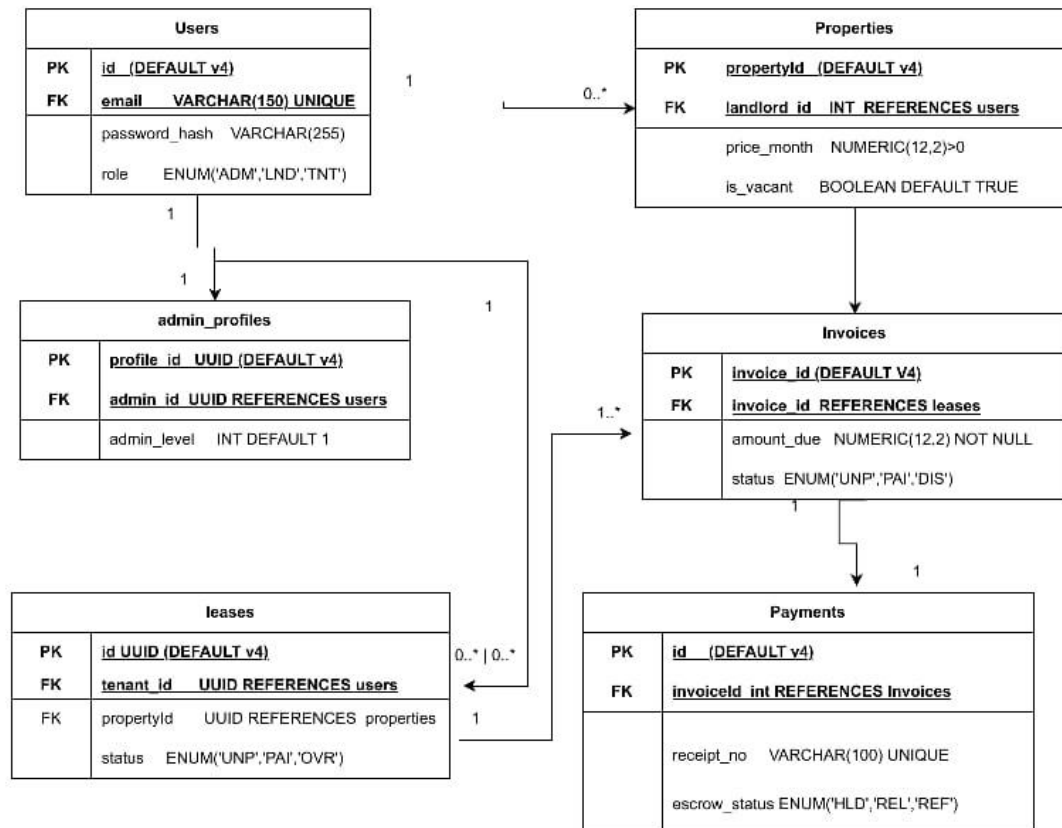


Objective: Takes the conceptual models and structures them into normalized tables with explicitly defined keys, independent of a specific database engine.

Explanation:

- This diagram introduces explicit Primary Keys (PK) and Foreign Keys (FK) relationships.
- The users table establishes a 1-to-many relationship with properties and leases.
- The structural bridge maps how a single lease references user IDs and property IDs. The leases table then connects downstream to invoices, which directly dictates the generation of payments.
- Data attributes use generic column definitions like NUMERIC prices and specific operational status enumerations (ENUM).

5.1.3.4 PHYSICAL ENTITY RELATIONSHIP DIAGRAM



Objective: The final implementation model showing actual target database data types, precise fields, and technical constraints.

Explanation:

- This shows exactly how tables are built inside the database software. Keys use distinct structural types like UUID (DEFAULT v4) or auto-incrementing integers (INT).
- Text inputs are assigned exact storage sizes like VARCHAR(150) for emails and VARCHAR(255) for passwords.
- The design introduces a concrete structural schema mapping for technical implementations, including tables like admin profiles that map exact administrative level tracking directly to database users.